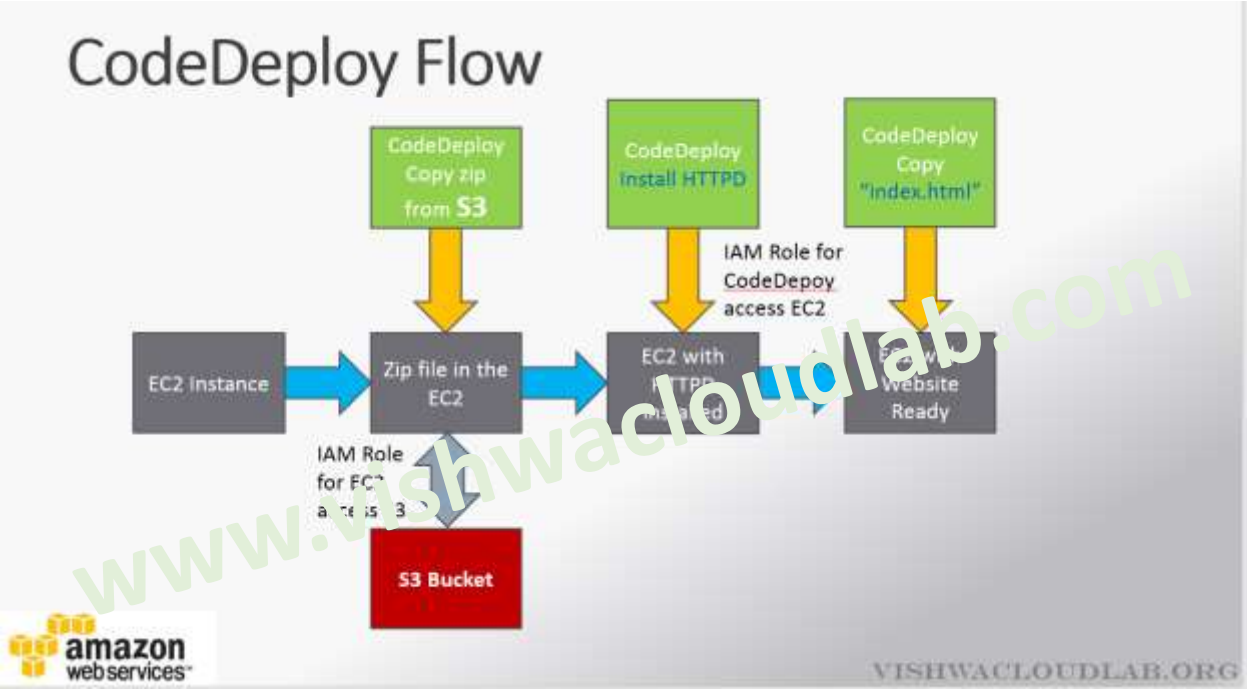
Objective of the LAB.

To deploy “http” service on 2 EC2 instance from CodeDeploy and also copy an test page to the EC2 instance.

Result:

We should have an web service running on the EC2 and we should be able to access it via the EC2 instance public IP.



Lab-Setups – CodeDeploy

1. Create IAM Policy & Role for EC2 to access S3

    1. Provide – S3 – get and list Permission for the EC2 instance. -- EC2-codedeploy

    Example for the name , You can have same name for Role and Policy,eg: -- EC2-codedeploy

2. Create Ec2 instance with “EC2-codedeploy” Roles attached to it and install the Codedeploy agent along with it (Bootstrap option)

3. Upload the “Sample-linuz.zip” into S3 bucket and give “Public” access

4. Create IAM Policy & Role for CodeDeploy to Access EC2 instance

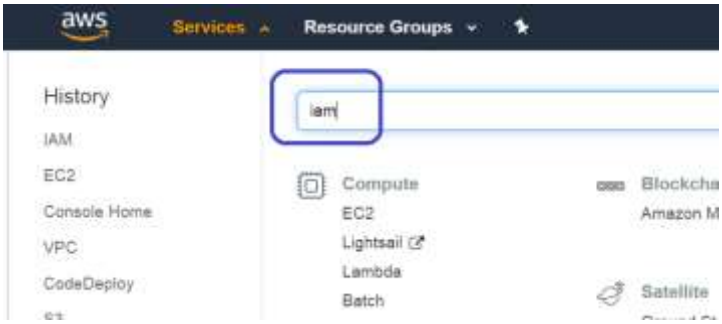
    Create Roles ,eg: -- and CodeDeploy

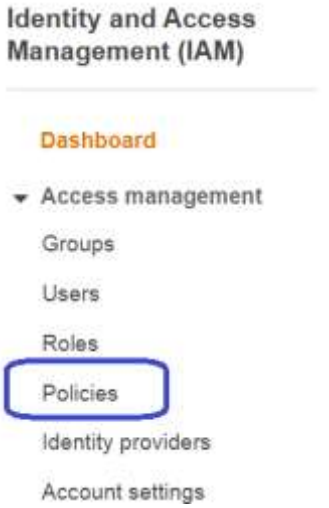
5. Create an CodeDeploy deployment and deploy the code.

6. Update the index.html file in the “.zip” file, and change “Congratulations” Edit the “appsepc” file. Remove all the lines from “hooks” and save it.

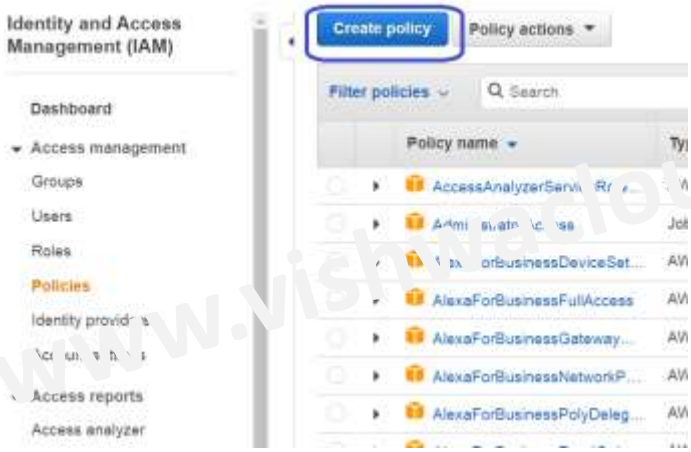
Step 1. Policy Creation

a. **IAM Policy** -- **EC2-codedeploy** for EC2 to access S3



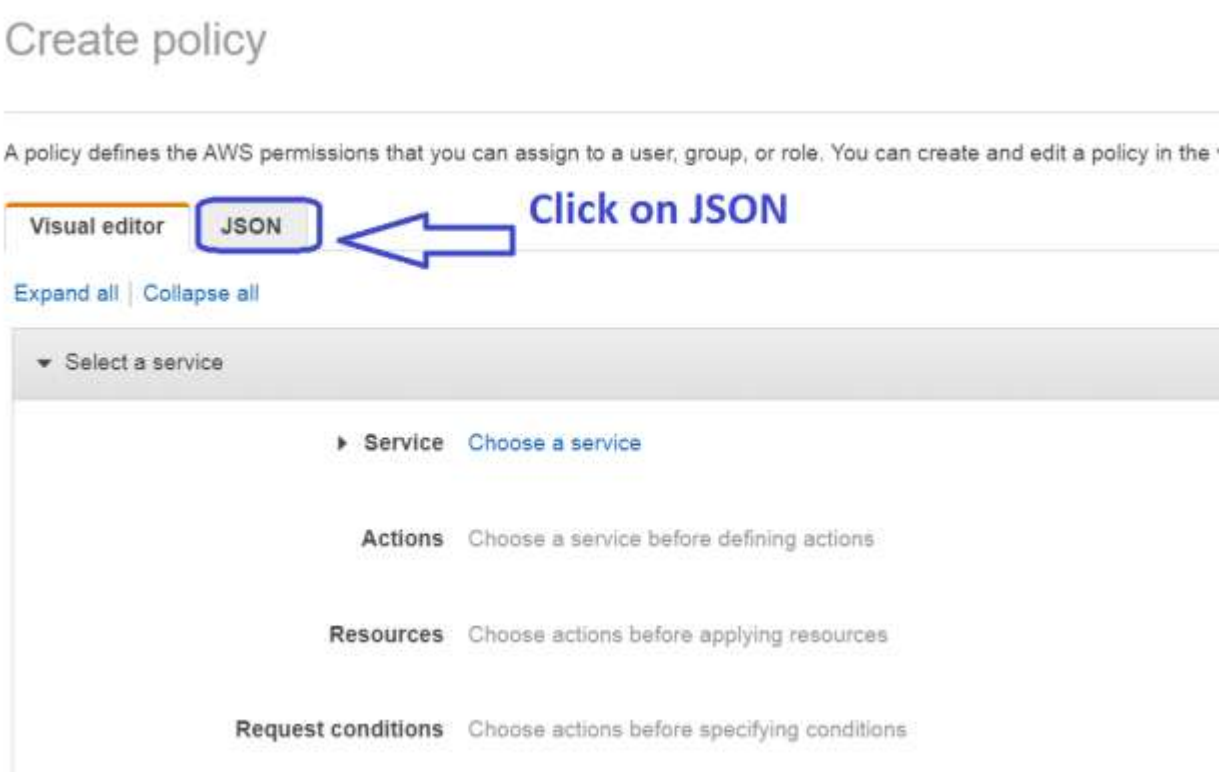


Click on **"Policies"**



Click on "Create Policy" to create an custom policy.

www.vishwacloudlab.com


Create policy
A policy defines the AWS permissions that you can assign to a user, group, or role. You can create and edit a policy in the
Visual editor
JSON
Click on JSON
Expand all | Collapse all
Select a service
Service
Choose a service
Actions
Choose a service before defining actions
Resources
Choose actions before applying resources
Request conditions
Choose actions before specifying conditions

Create policy
A policy defines the AWS permissions that you can assign to a user, group, or role.
Visual editor
JSON
{
"Version": "2012-10-17",
"Statement": []
}

Copy the below json content into the policy and save it.

```
{
	"Version": "2012-10-17",
	"Statement": [
		{
			"Action": [
				"s3:Get*",
				"s3:List*"
			],
			"Effect": "Allow",
			"Resource": "*"
		}
	]
}
```

Create policy

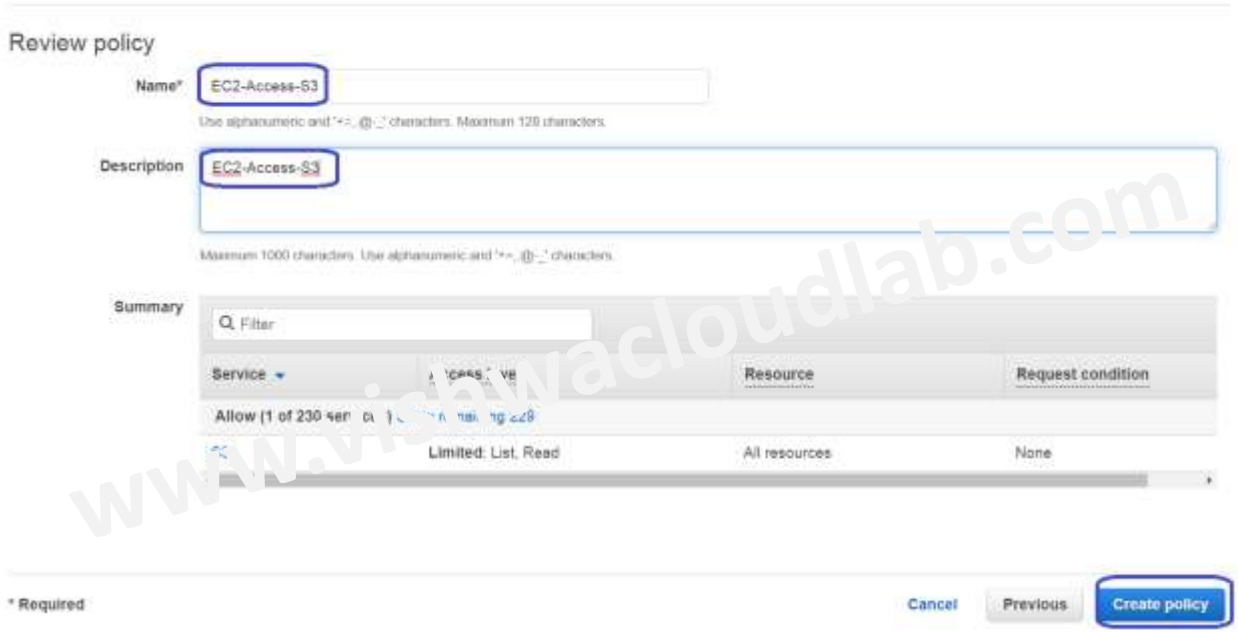
A policy defines the AWS permissions that you can assign to a user, group, c

Visual editor | JSON

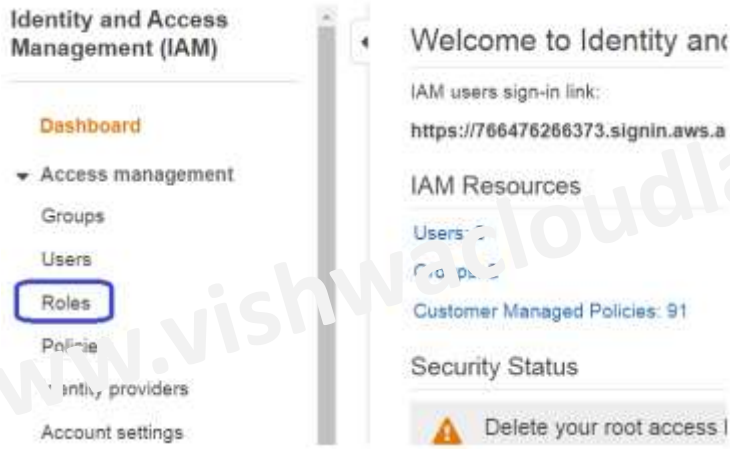
```
1  {
2      "Version": "2012-10-17",
3      "Statement": [
4          {
5              "Action": [
6                  "s3:Get*",
7                  "s3:List*"
8              ],
9              "Effect": "Allow",
10             "Resource": "*"
11         }
12     ]
13 }
14
```

www.vishwacloudlab.com

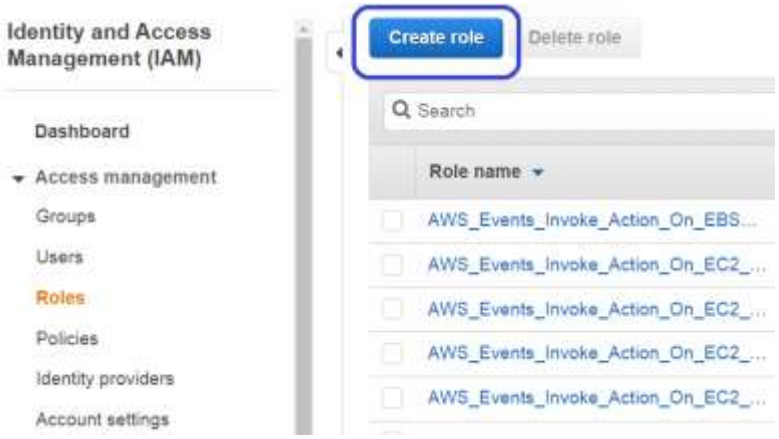
Click on “Review Policy”

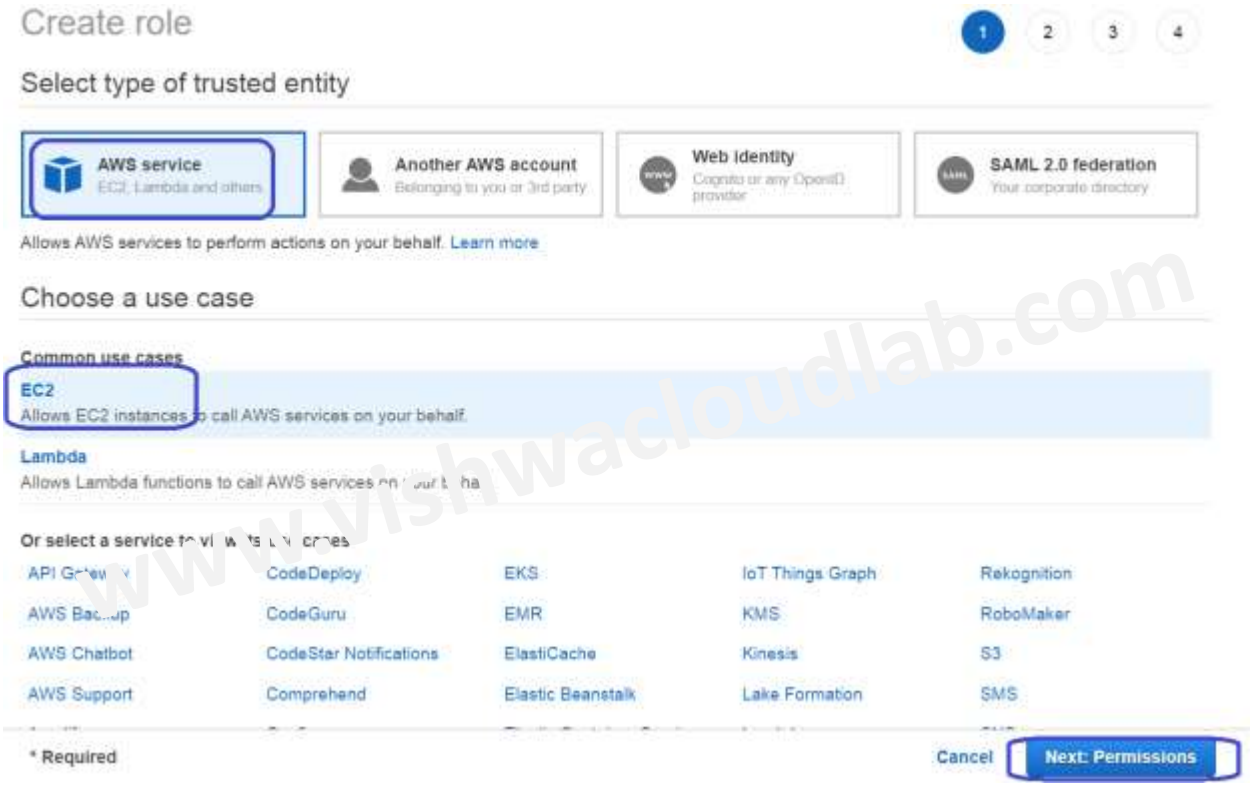


b. IAM Role for **EC2 Access the S3**

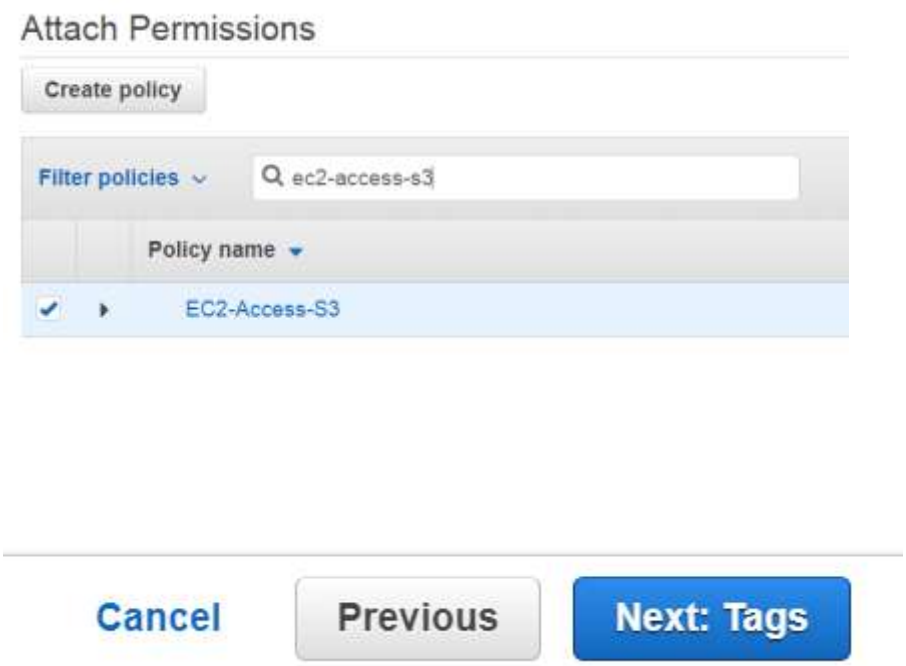


Click on “Roles”

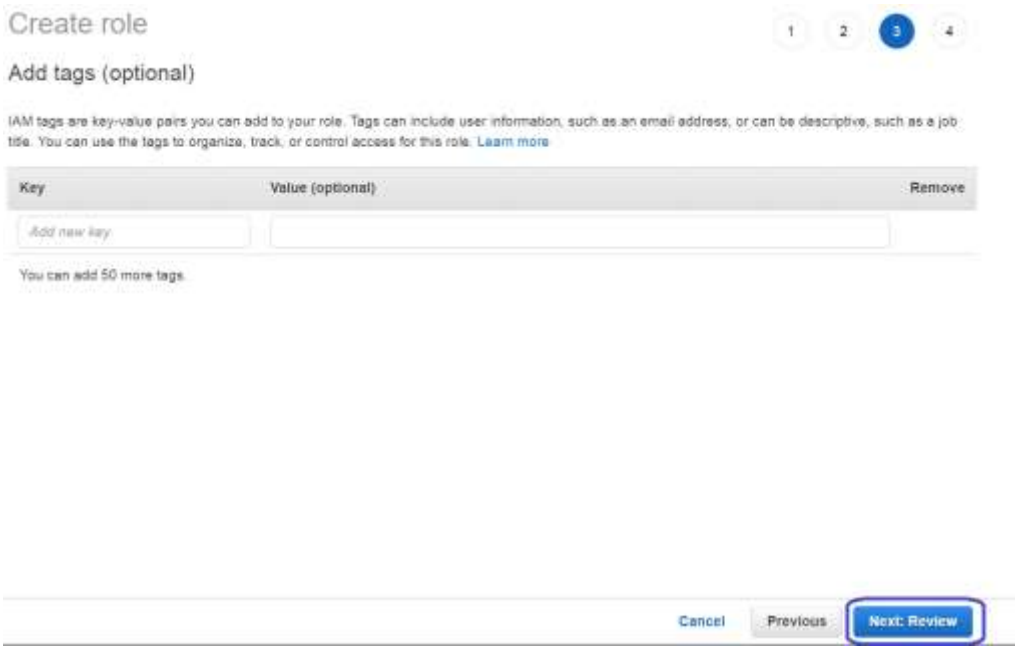




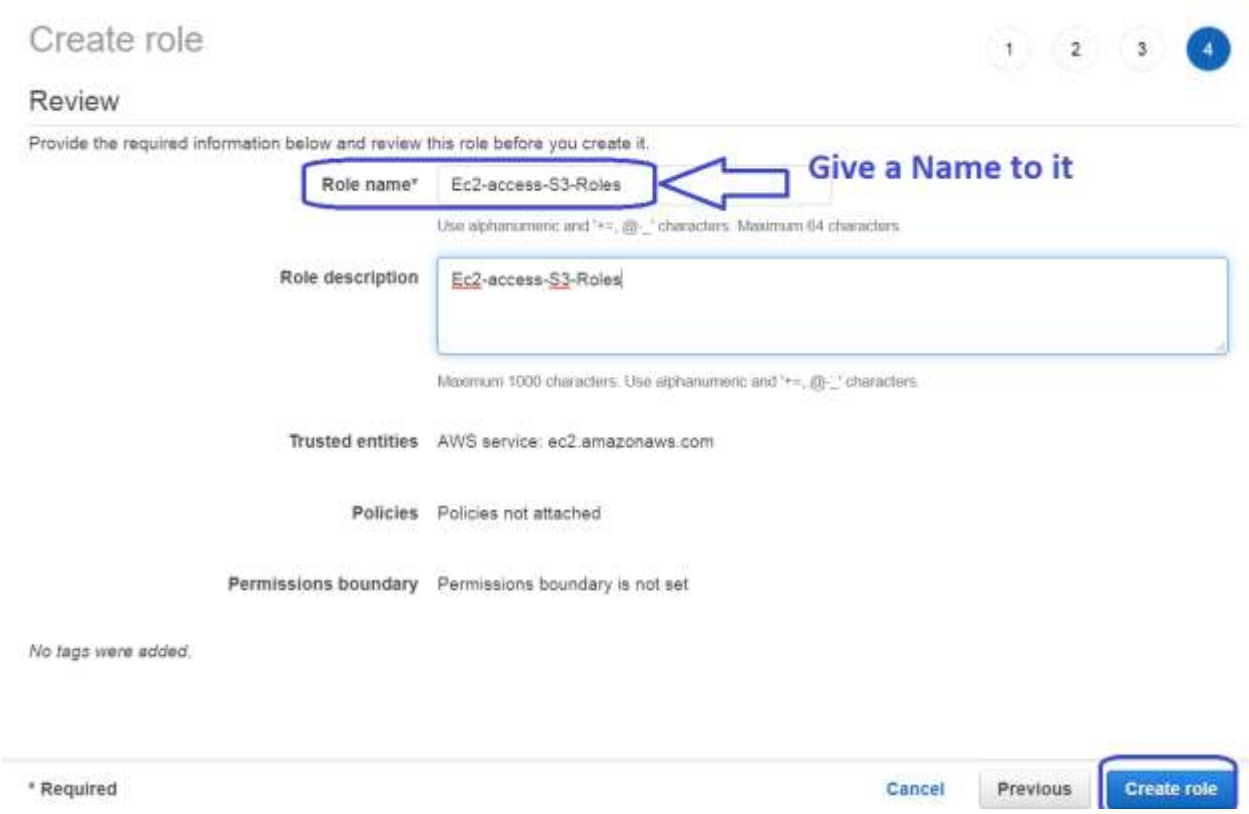
Click on **"Next:Permissions"**.



Click on “next”.



Click on **“Review”**

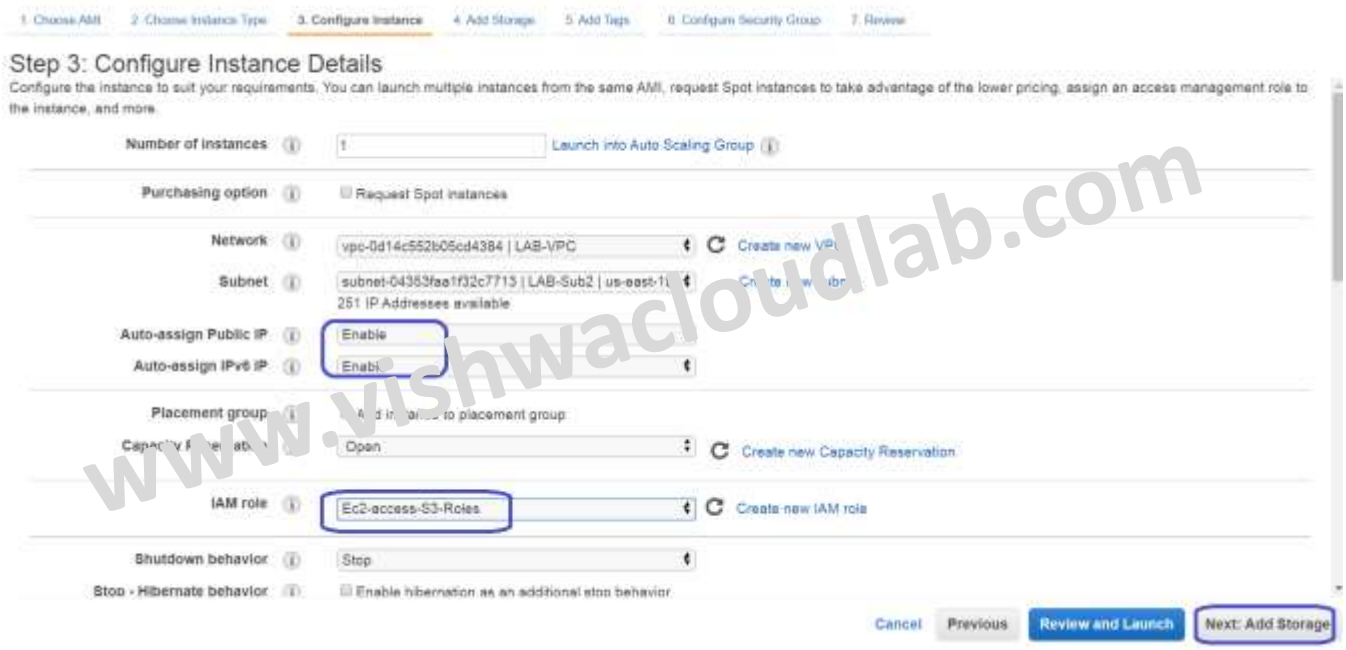


Click on **"Create Role"**

Step 2. Create EC2 instance

Create 2 Instance with the Above IAM role that was created for EC2 "**EC2-codedeploy**"

a.



www.vishwacloudlab.com

Select the IAM roles while creating the EC2 instance.

b.

Under the Advance option copy the Below command along with it

www.vishwacloudlab.com

```
#!/bin/bash
sudo yum install ruby -y
sudo yum install wget -y
cd /home/ec2-user
wget https://aws-codedeploy-us-east-1.s3.amazonaws.com/latest/install
chmod +x ./install
sudo ./install auto
```

These above codes needs to be copied under the advance section in the **"userdata"**, which would automatically install the Codedeploy agent and its called as "**Bootstrapping**"

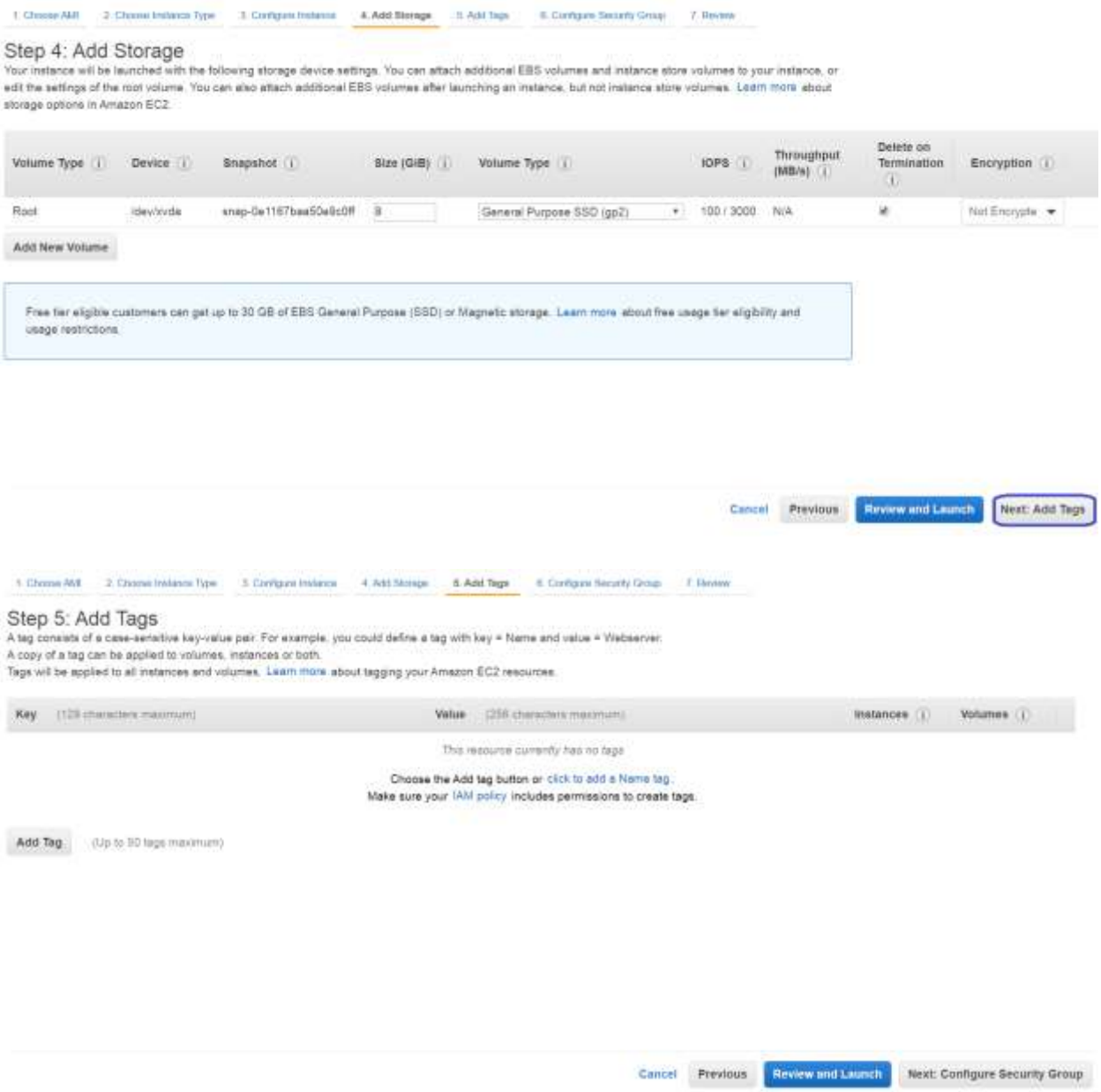
The heighted portion in the http link could be replaced with the below code, based on the region selected.

Resource Kit Bucket Names by Region

This table lists the names of `bucket-name` replacements required for some procedures in the guide. These are the names of the Amazon S3 buckets that contain the AWS CodeDeploy Resource Kit files.

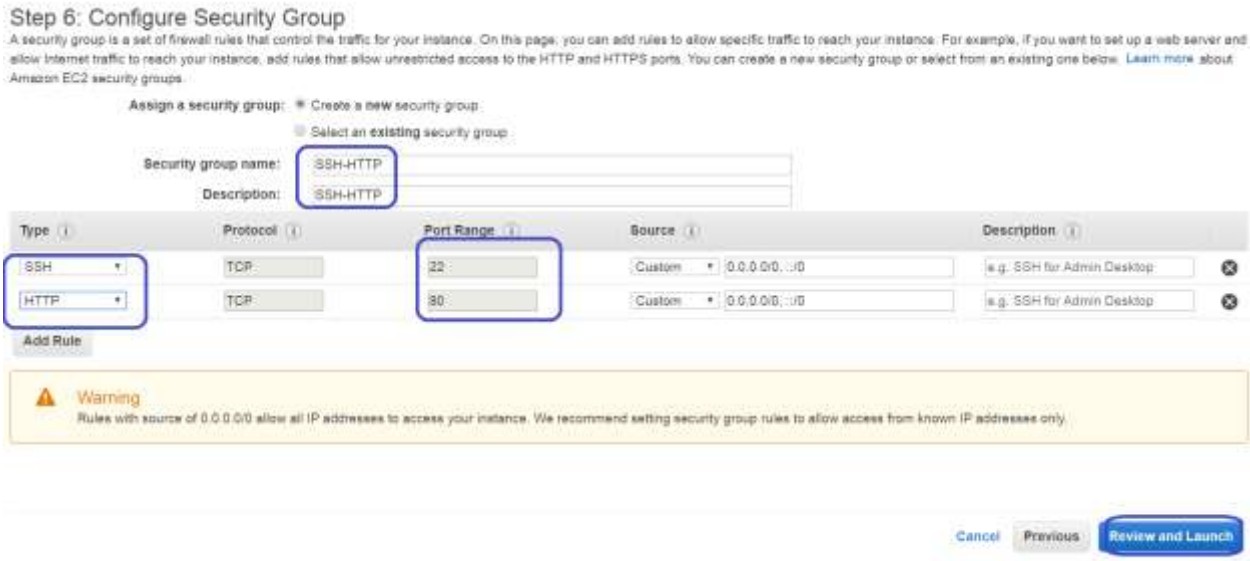
Region name	`bucket-name` replacement	Region identifier
US East (Ohio)	aws-codedeploy-us-east-2	us-east-2
US East (N. Virginia)	aws-codedeploy-us-east-1	us-east-1
US West (N. California)	aws-codedeploy-us-west-1	us-west-1
US West (Oregon)	aws-codedeploy-us-west-2	us-west-2
Canada (Central)	aws-codedeploy-ca-central-1	ca-central-1
EU (Ireland)	aws-codedeploy-eu-west-1	eu-west-1
EU (London)	aws-codedeploy-eu-west-2	eu-west-2
EU (Paris)	aws-codedeploy-eu-west-3	eu-west-3
EU (Frankfurt)	aws-codedeploy-eu-central-1	eu-central-1
Asia Pacific (Tokyo)	aws-codedeploy-ap-northeast-1	ap-northeast-1
Asia Pacific (Seoul)	aws-codedeploy-ap-northeast-2	ap-northeast-2
Asia Pacific (Singapore)	aws-codedeploy-ap-southeast-1	ap-southeast-1
Asia Pacific (Sydney)	aws-codedeploy-ap-southeast-2	ap-southeast-2
Asia Pacific (Mumbai)	aws-codedeploy-ap-south-1	ap-south-1
South America (São Paulo)	aws-codedeploy-sa-east-1	sa-east-1

c. Click next on the “Add storage”

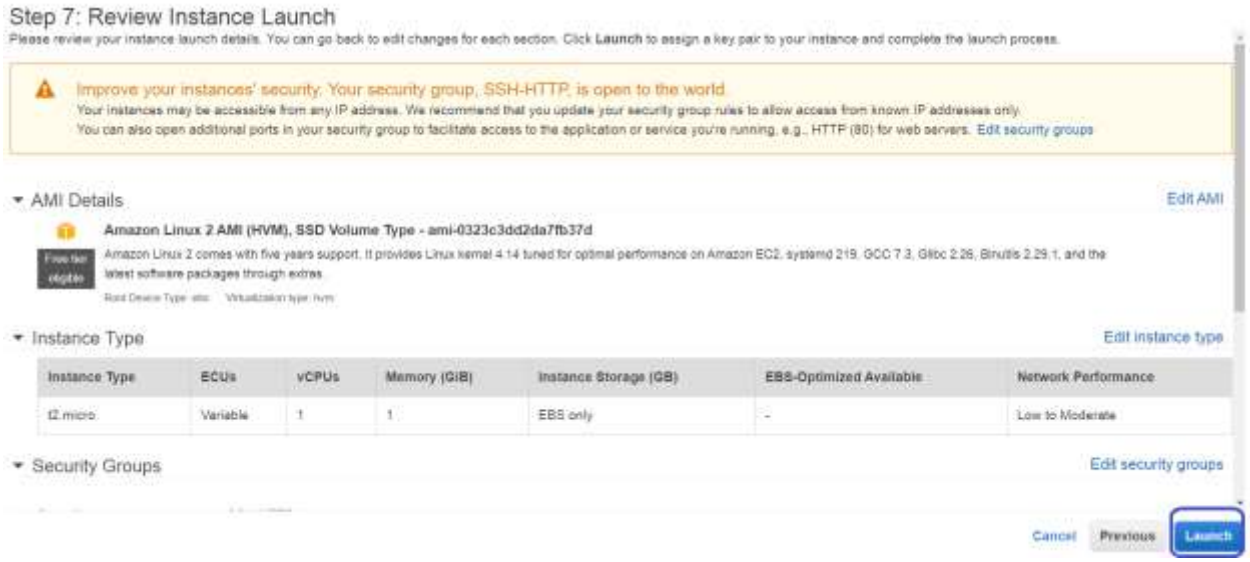


Click on “next”

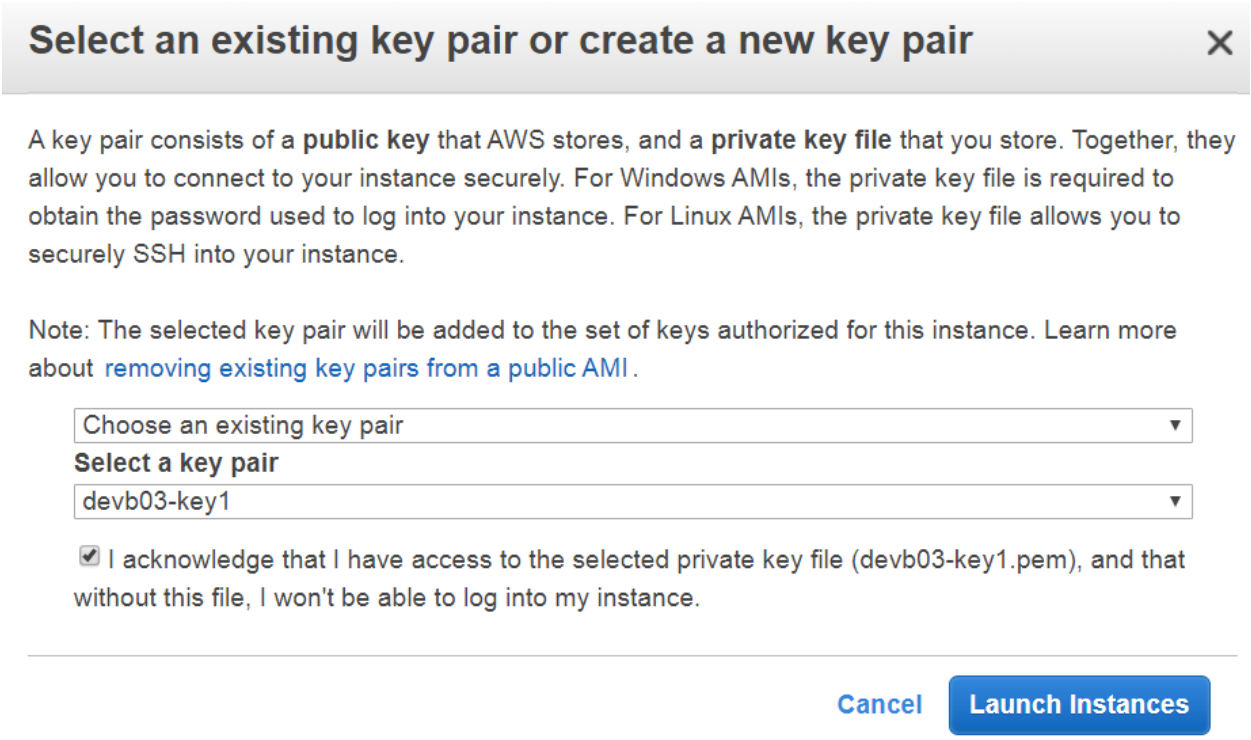
d. With Security Group allowing -- SSH and HTTP traffic



Click on “review and launch”



Click on “Launch”



We would need a key to login to the ec2 instance later.

Here the key is already created and the ".pem" file is already downloaded on the system.

Click on "launch".

For more info, on creating the EC2 and how to login, refer to the "EC2-instance creation" and "Login to EC2 via xshell and putty" document.

e. After the EC2 instance is up, check if the codedeploy agent service is running with the below command.

 # service codedeploy-agent status.

Output:

```
[root@ip-172-31-83-93 ec2-user]# service codedeploy-agent status
The AWS CodeDeploy agent is running as PID 2704
[root@ip-172-31-83-93 ec2-user]#
```

www.vishwacloudlab.com

Step 3.

Upload the “Sample-linuz.zip” into S3 bucket and give “Public” access.

The bucket needs to have "Versioning" ENABLED.

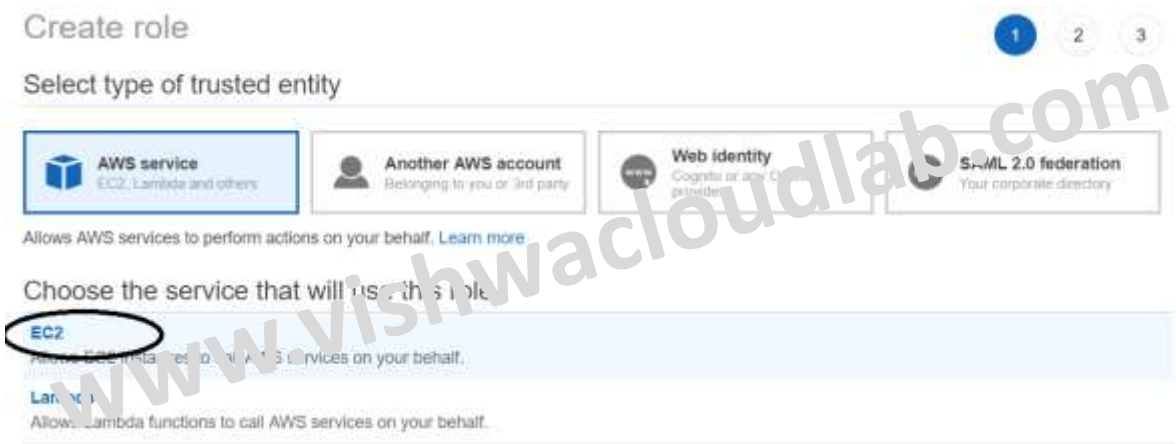
The file is available in the Google drive
https://drive.google.com/drive/u/1/folders/18wNY2GTIcS3vCufomVo9If5B59P5l8nS

Step 4: Create IAM Role and Policy for **CodeDeploy**

a. IAM Policy -- **Codedeploy** to access EC2 instance.
Eg:-- "**EC2-CodeDeploy**"

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "VisualEditor0",
            "Effect": "Allow",
            "Action": [
                "ec2:*"
            ],
            "Resource": "*"
        }
    ]
}
```

IAM Role for **"EC2-CodeDeploy-R"**



www.vishwacloudlab.com

Then attach the above Codedeploy Policy to this IAM role

Add the below trust relationship on the CodeDeploy Role

Click on “Edit trust relationship”

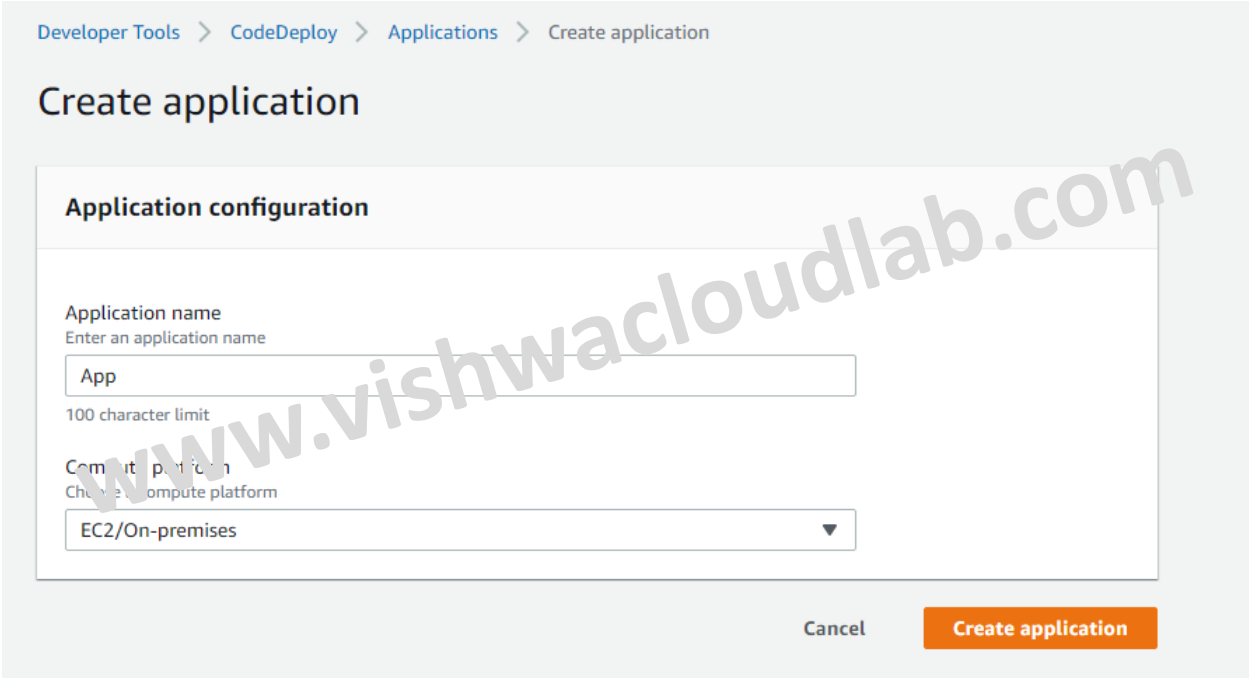
```
{
   "Version": "2012-10-17",
   "Statement": [
      {
         "Sid": "",
         "Effect": "Allow",
         "Principal": {
            "Service": [
               "codedeploy.us-east-1.amazonaws.com",
               "codedeploy.us-west-2.amazonaws.com"
            ]
         },
         "Action": "sts:AssumeRole"
      }
   ]
}
```

Step 5: Create an CodeDeploy deployment and deploy the code.

a.

Click on “Create Application”

b.



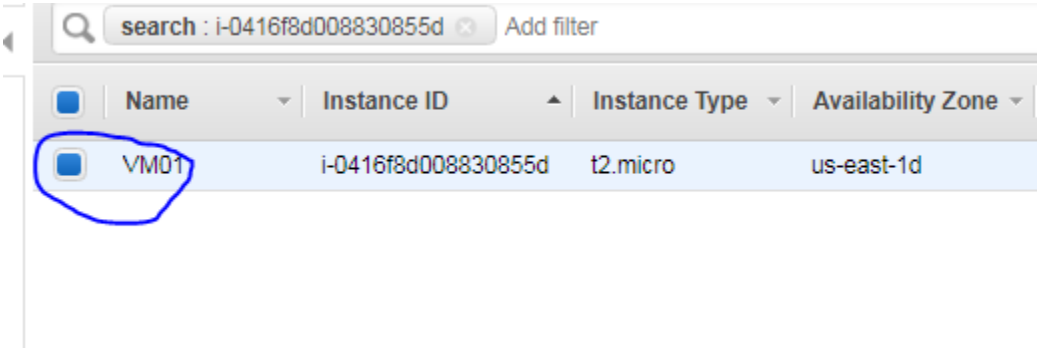
www.vishwacloudlab.com

Click on “create Application”

c.

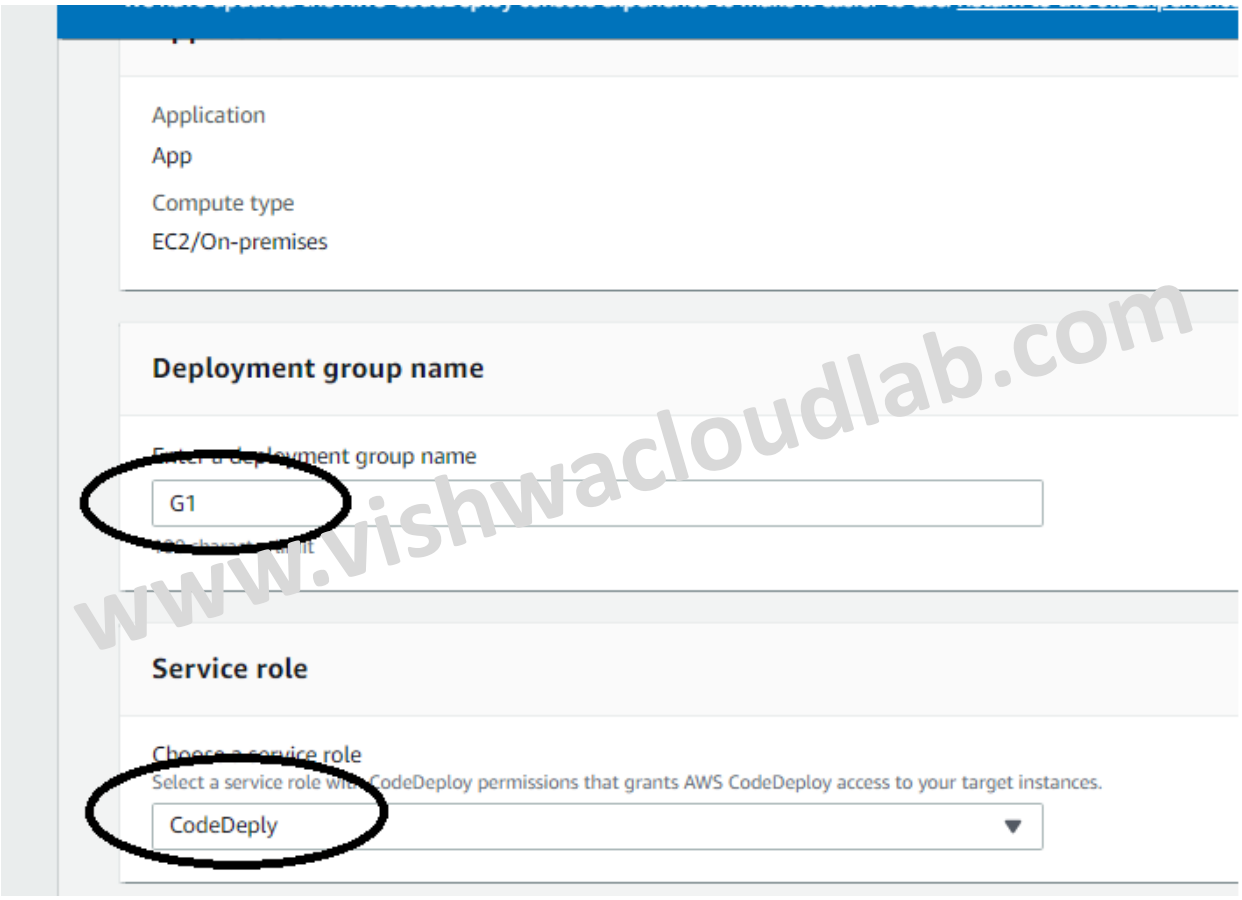
Click on “Create deployment group”

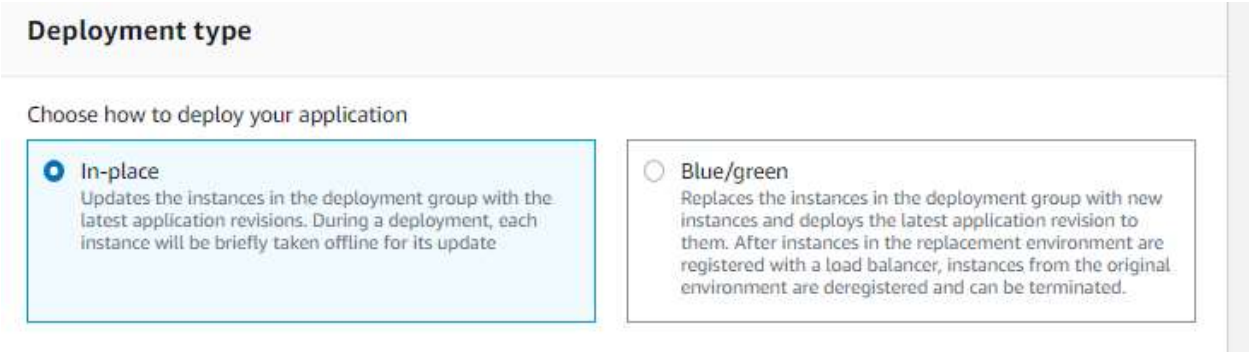
Note: -- Make sure you have a tag attached to your EC2 instance.



www.vishwacloudlab.com

d. Create a Deployment Group





www.vishwacloudlab.com

Environment configuration

Select any combination of Amazon EC2 Auto Scaling groups, Amazon EC2 instances, and on-premises instances to add to this deployment

☐ Amazon EC2 Auto Scaling groups

☑ Amazon EC2 instances
1 unique matched instance. Click here for details

You can add up to three groups of tag filters to Amazon EC2 instances to this deployment group.
One tag group: Any instances identified by the tag group will be deployed to.
Multiple tag groups: Only instances identified by all the tag groups will be deployed to.

Tag group 1

Key	Value - *optional*	
Name	VM01	Remove tag

Add tag

+ Add tag group

Deployment settings

Deployment configuration
Choose from a list of default and custom deployment configurations. A deployment configuration is a set of rules that determines how fast an application will be deployed and the success or failure conditions for a deployment.

CodeDeployDefault.AllAtOnce ▼ or Create deployment configuration

Load balancer

Select a load balancer to manage incoming traffic during the deployment process. The load balancer blocks traffic from each instance while it's being deployed to and allows traffic to it again after the deployment succeeds.

☐ Enable load balancing

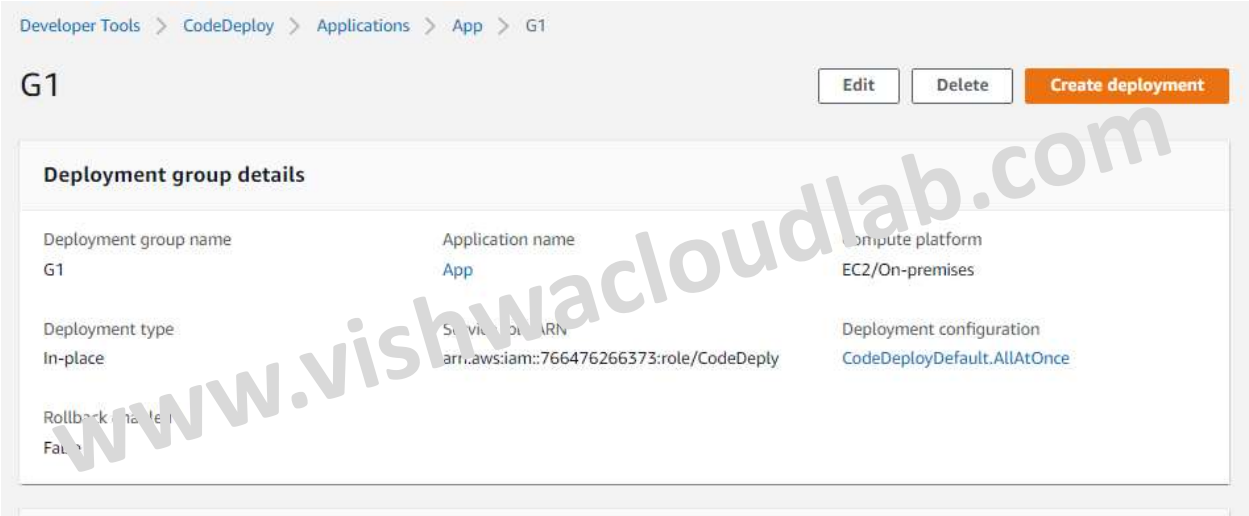
▸ Advanced - optional

Cancel Create deployment group

www.vishwacloudlab.com

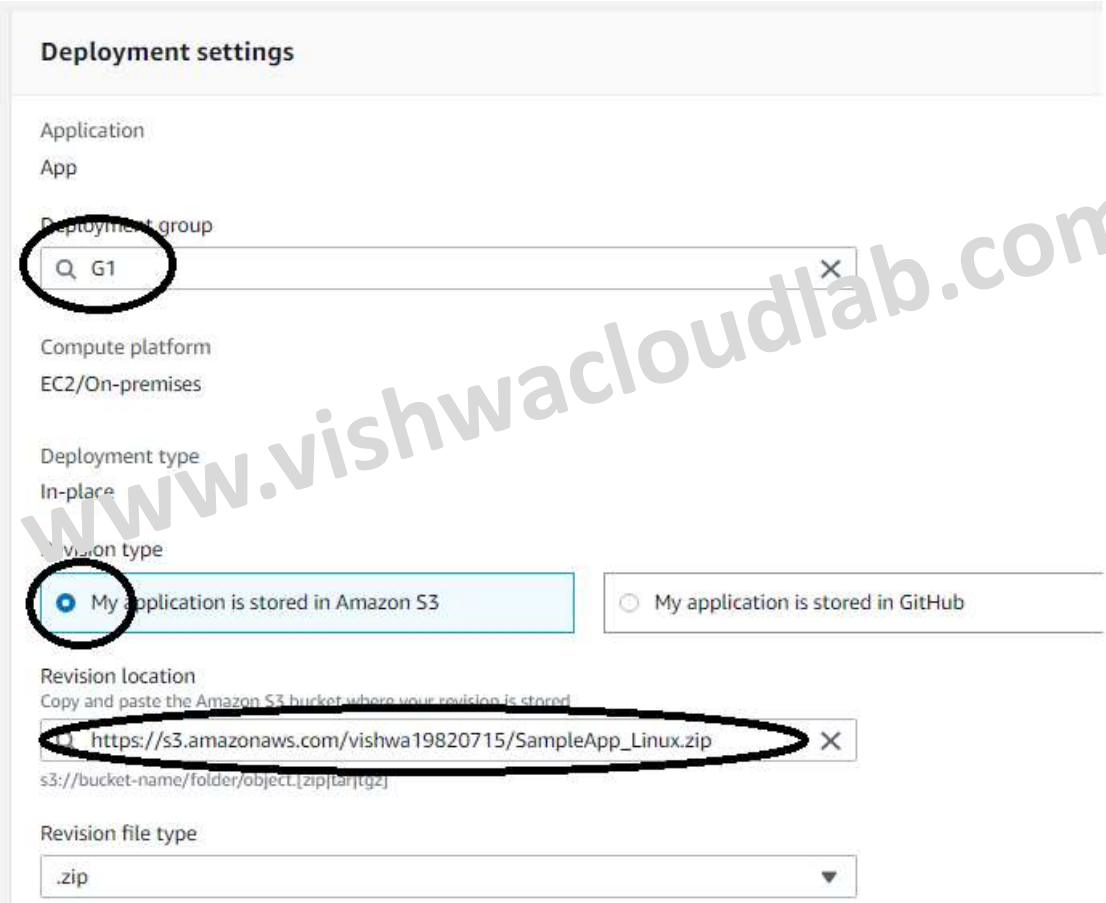
Make sure to “untick” the Loadbalancer.

Output:

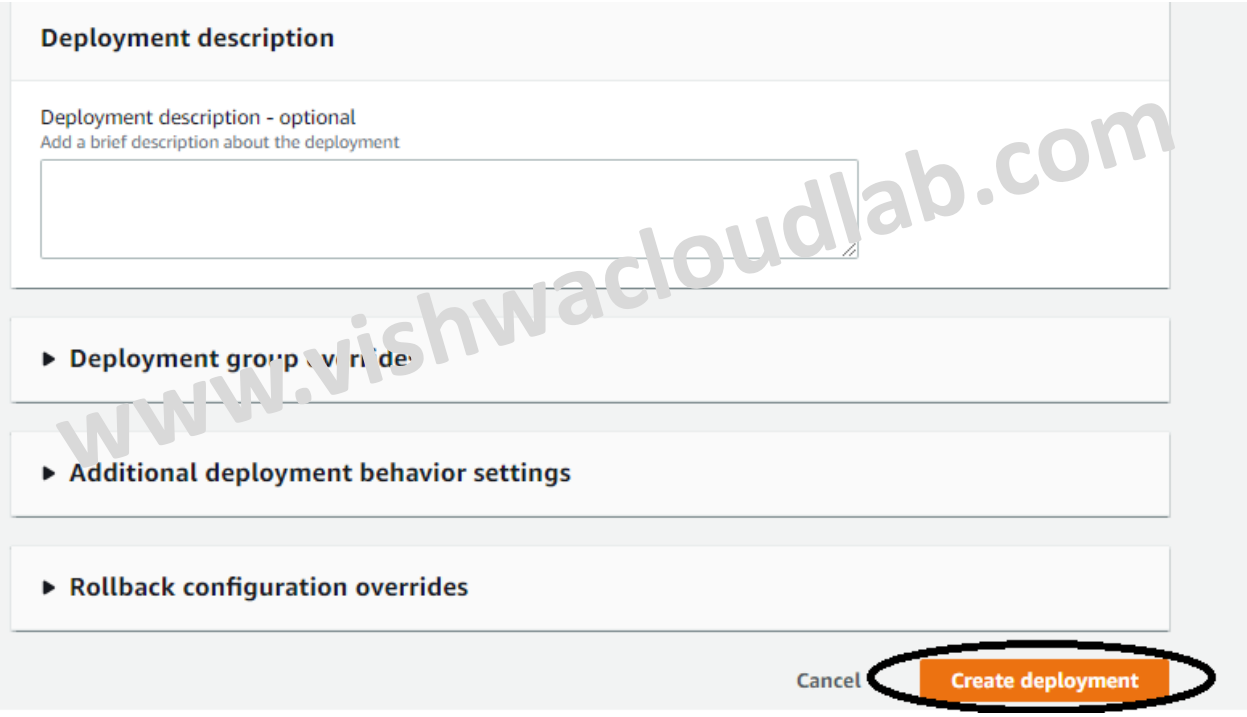


www.vishwacloudlab.com

e. Create Deployment



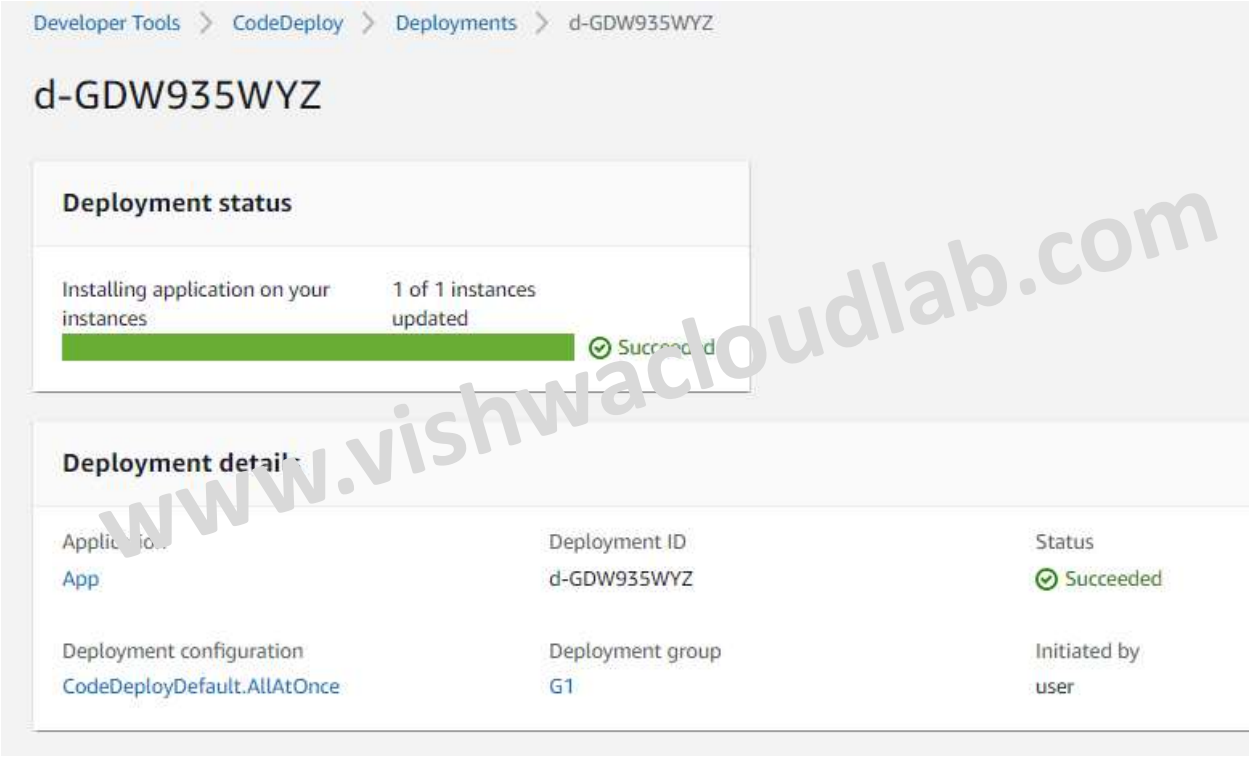
www.vishwacloudlab.com



www.vishwacloudlab.com

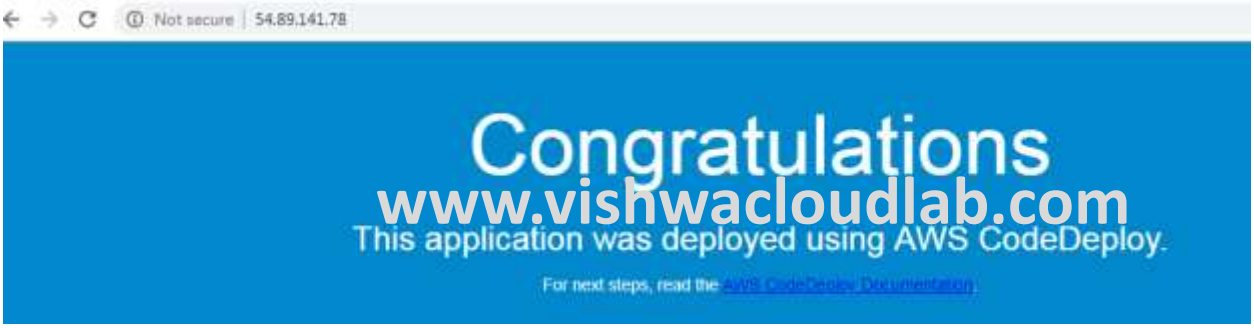
Click on “Create Deployment”

Output:



This show that the deployment is successful.

Now lets try to access the webpage with public ip of the EC2 instance.



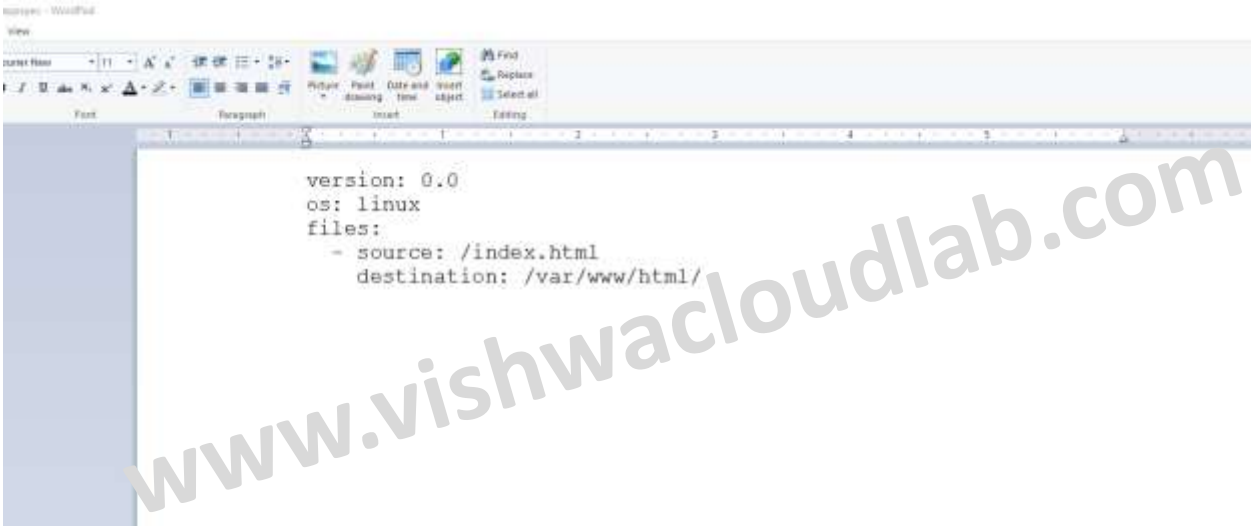
www.vishwacloudlab.com

Step 6: Update the Deploy with new content

Download the Zip file on your laptop, uncompress it to a folder.

Remove the “Scripts” folder.

Edit the appspec file as below. Remove all other lines.



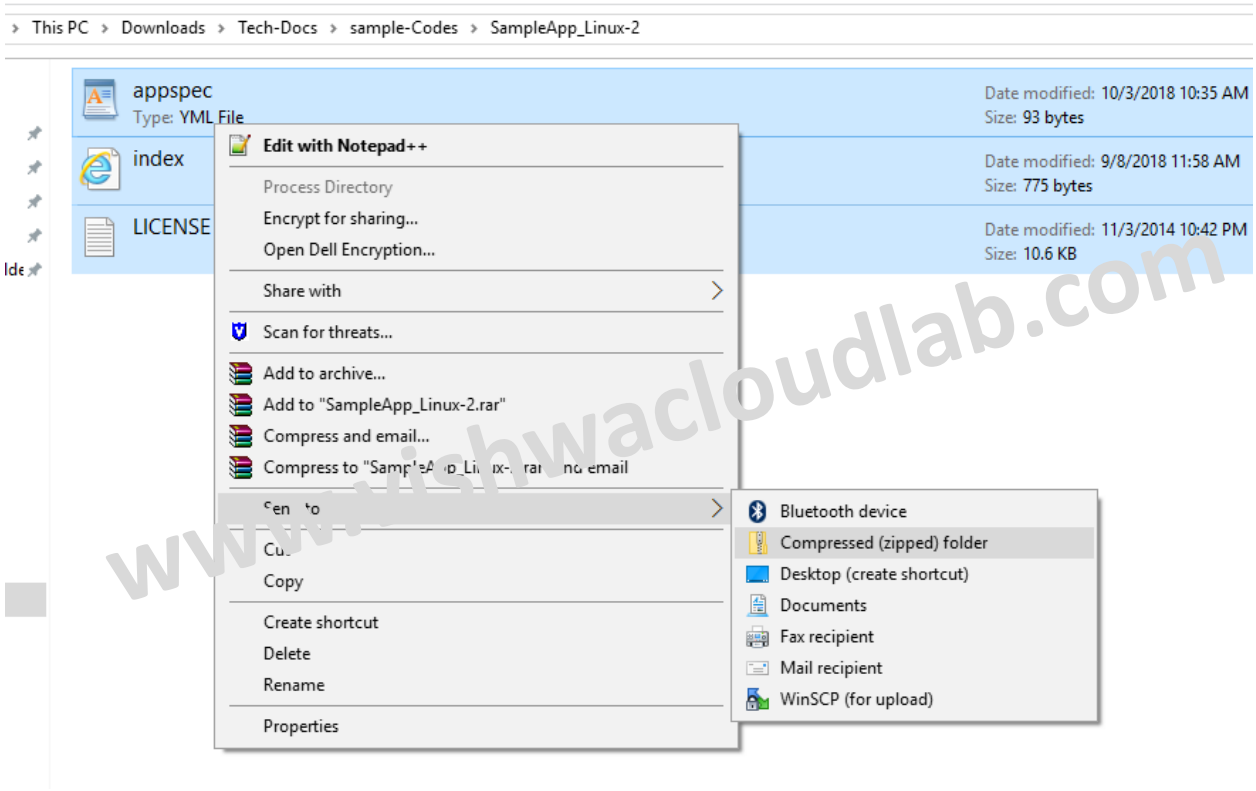
```
version: 0.0
os: linux
files:
  - source: /index.html
    destination: /var/www/html/
```

Now edit the “index.html” file. And add some content as below. And save it.

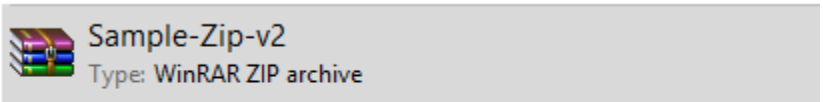
```
index.html
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Sample Deployment</title>
  <style>
    body {
      color: #ffffff;
      background-color: #0188cc;
      font-family: Arial, sans-serif;
      font-size: 14px;
    }

    h1 {
      font-size: 500%;
      font-weight: normal;
      margin-bottom: 0;
    }

    h2 {
      font-size: 200%;
      font-weight: normal;
      margin-bottom: 0;
    }
  </style>
</head>
<body>
  <div align="center">
    <h1>Congratulations Batch, HTTP service is UP</h1>
    <h2>This application was deployed using AWS CodeDeploy. But was customized by Vishwa</
    <p>For next steps, read the <a href="http://aws.amazon.com/documentation/codedeploy">A
    Documentation</a>.</p>
  </div>
</body>
</html>
```

www.vishwacloudlab.com

Now zip the file as shown. Select only the FILES and zip and NOT the folder itself, as CODEDEPLOY would be looking for “appsepc” file in the zip parent folder.



www.vishwacloudlab.com



Now upload this Sample-zip-v2 to S3, and make it public.

Update the same Deployment Group with “new Revision” with new S3 link.